

## 8. Introduction

---

Almost every non-trivial program eventually needs to answer one question fast: *"Given this name, what's the record?"* A username to a profile. A product ID to a price. An error code to a message. How you answer that question decides whether your code feels snappy or sluggish.

In Python, the tool built for exactly this is the **dictionary**. It's one of the most-used data structures in real software, and it's a favorite of interviewers because it quietly tests whether you understand *cost*. Anyone can loop through a list. Fewer people can explain why a dictionary finds a value in roughly one step no matter how much data you throw at it.

This lesson builds that understanding. We'll cover what keys and values actually are, how to read a dictionary without crashing your program, how to loop over pairs, how to nest dictionaries to model real records, and the classic bugs that trip up beginners. By the end, you'll know not just *how* to use a dict, but *when* and *why*.

If you're learning **Python**, preparing for a **coding interview**, or strengthening your grasp of core **data structures**, this is a foundational stop.

## 9. Problem Overview

---

Think of a coat check at a theater. You hand over a ticket, and the attendant walks straight to your coat and hands it back. They don't inspect every coat on the rack — the ticket *is* the location.

A Python dictionary works the same way. Instead of storing values by **position** (like a list, where item 0 comes before item 1), it stores them under a **name of your choosing**, called a *key*. Ask for the key, and Python jumps straight to the matching *value*.

Formally, a dictionary is a collection of **key–value pairs** with three defining rules:

- Each **key is unique** — one label per locker.
- Keys must be **immutable** (unchangeable) — strings, numbers, or tuples, never lists.
- Lookup by key is **near-instant** and stays fast whether you have ten entries or ten million.

That last property is the headline feature, and it's what we'll unpack most carefully.

## 10. Example

---

Let's model people and their ages.

```
ages = {"Sam": 20, "Mia": 25}

print(ages["Sam"])    # 20
print(ages["Mia"])    # 25
```

Here "Sam" and "Mia" are keys; 20 and 25 are values. Writing `ages["Sam"]` tells Python: *"go to the locker labeled Sam and give me what's inside."* The output is 20 because that's the value stored under that key — no scanning required.

Now add a new person:

```
ages["Leo"] = 30
print(ages)    # {'Sam': 20, 'Mia': 25, 'Leo': 30}
```

Notice there's no special "add" command. Assigning to a key that doesn't exist **creates** it. Assigning to a key that already exists **replaces** it:

```
ages["Sam"] = 99
print(ages["Sam"])    # 99 — the old 20 is gone
```

That's the uniqueness rule in action. One label, one locker.

## 11. Intuition

---

Here's how an experienced engineer thinks about this.

When you store data in a **list**, the only way to find "the record for Sam" is to walk from the front, checking each item until you hit a match. With three items that's cheap. With ten million, you might scan all ten million. The cost grows with the data — that's **O(n)**.

A dictionary refuses to search at all. The trick is a technique called **hashing**. When you use a key like "Sam", Python runs it through a *hash function* — a small piece of math that turns the key into a number. That number tells Python *which slot in memory* the value lives in. So instead of scanning, Python computes the location directly and jumps there.

This is why the key must be **immutable**. If a key could change after it was stored, its hash would change too, and Python would compute a *different* slot next time — it would look in the wrong locker and never find your value. A stable key means a stable address.

Once you internalize "the key computes the address," the whole data structure clicks. Lookup doesn't get slower as data grows, because computing an address takes the same effort every time. That constant cost is  **$O(1)$**  — the reason engineers reach for dictionaries constantly.

## 12. Brute Force Solution

---

Before dictionaries, how would you build a lookup with just a **list**?

**Idea:** Store each record as a pair inside a list, then scan the list to find a match.

```
people = [("Sam", 20), ("Mia", 25), ("Leo", 30)]

def find_age(name):
    for person_name, age in people:    # walk every item
        if person_name == name:
            return age
    return None
```

**Algorithm:** 1. Loop through every element. 2. Compare each element's name to the target. 3. Return the value on a match; return `None` if the loop ends.

**Advantages:** - Simple, no special structure needed. - Preserves insertion order naturally. - Fine for tiny datasets.

**Disadvantages:** - Every lookup may scan the entire list. - Gets painfully slow as data grows. - No protection against duplicate names.

**Complexity:** - **Time:**  $O(n)$  per lookup — cost scales with the number of records. - **Space:**  $O(n)$  to store the records.

This works, but it does exactly the "rummaging through the rack" we're trying to avoid.

## 13. Optimal Solution

---

Replace the list with a dictionary and the scan disappears.

```
people = {"Sam": 20, "Mia": 25, "Leo": 30}

age = people["Sam"]    # direct jump — no loop
```

Step by step, here's what happens on `people["Sam"]`:

**Step****What Python does**

- 1 Runs "Sam" through the hash function → a number
- 2 Uses that number to compute a memory slot
- 3 Jumps directly to that slot
- 4 Returns the stored value 20

Compare the search cost visually:

```
LIST lookup:  [box0] → [box1] → [box2] → ... → match    (scan)
DICT lookup:  "Sam" → hash → slot → value                (jump)
```

The list's arrows grow with your data. The dict's path is the same length every time. That's the entire advantage in one picture.

**Reading safely with get()**

Asking for a missing key with square brackets **crashes** your program:

```
people["Zoe"]    # KeyError: 'Zoe' – stops execution
```

The `get()` method is the gentle alternative. It returns `None` (or a fallback you choose) instead of raising:

```
people.get("Zoe")        # None
people.get("Zoe", 0)     # 0 – your chosen default
```

Use `get()` whenever you're not certain the key exists.

**14. Python Code**

```
def build_directory(pairs):
    """Turn a list of (name, age) pairs into a fast lookup by name."""
    directory = {}
    for name, age in pairs:
        directory[name] = age          # new key creates, existing key overwrites
    return directory

def lookup_age(directory, name, default=None):
    """Read a value safely – never crashes on a missing name."""
    return directory.get(name, default)
```

```
def main():
    people = build_directory([("Sam", 20), ("Mia", 25), ("Leo", 30)])

    # Direct, near-instant lookup
    print(lookup_age(people, "Sam"))          # 20
    print(lookup_age(people, "Zoe", 0))       # 0 (safe default)

    # Loop over key/value pairs together
    for name, age in people.items():
        print(f"{name} is {age}")

    # Nested dictionaries model real records
    users = {"sam": {"age": 20, "city": "Delhi"}}
    print(users["sam"]["city"])               # Delhi

    # Everyday helpers
    print("Mia" in people)                    # True
    print(len(people))                       # 3
    del people["Leo"]
    print(people)                            # {'Sam': 20, 'Mia': 25}

if __name__ == "__main__":
    main()
```

## 15. Code Walkthrough

- `directory = {}` — an empty dictionary. Curly braces with nothing inside.
- `directory[name] = age` — the create-or-overwrite rule. There's no separate "add" method; assignment handles both.
- `directory.get(name, default)` — safe read. Returns the value if present, otherwise `default`. This is what keeps `lookup_age` from ever throwing a `KeyError`.
- `people.items()` — hands back each key and value *together*, so we unpack them into `name`, `age` right in the `for` line. Looping over a dict *without* `.items()` gives only the keys — a common surprise.
- `users["sam"]["city"]` — chained keys. First `users["sam"]` returns the inner record `{"age": 20, "city": "Delhi"}`, then `["city"]` reads from it. This is exactly how JSON data from websites and APIs is shaped.
- `"Mia" in people` — checks for a key's existence before you touch it, avoiding crashes.
- `len(people)` — counts the pairs.
- `del people["Leo"]` — removes a pair entirely.

## 16. Dry Run

Let's trace `build_directory([("Sam", 20), ("Mia", 25), ("Leo", 30)])`:

| Iteration | name, age | Action           | directory after                                |
|-----------|-----------|------------------|------------------------------------------------|
| start     | —         | —                | <code>{}</code>                                |
| 1         | Sam, 20   | key new → create | <code>{'Sam': 20}</code>                       |
| 2         | Mia, 25   | key new → create | <code>{'Sam': 20, 'Mia': 25}</code>            |
| 3         | Leo, 30   | key new → create | <code>{'Sam': 20, 'Mia': 25, 'Leo': 30}</code> |

Now `lookup_age(directory, "Sam")` runs `directory.get("Sam", None)`. Python hashes "Sam", jumps to its slot, finds 20, and returns it — no iteration over the other keys at all.

Then `lookup_age(directory, "Zoe", 0)`: hashing "Zoe" lands on an empty slot, so `get` returns the default 0 instead of crashing.

## 17. Complexity Analysis

| Operation                                        | Time                | Why                                      |
|--------------------------------------------------|---------------------|------------------------------------------|
| Lookup <code>d[k]</code> / <code>d.get(k)</code> | <b>O(1)</b> average | Hash computes the slot directly; no scan |
| Insert <code>d[k] = v</code>                     | <b>O(1)</b> average | Same hashing path to find the slot       |
| Delete <code>del d[k]</code>                     | <b>O(1)</b> average | Locate slot, clear it                    |
| <code>k in d</code>                              | <b>O(1)</b> average | Just a lookup that returns a boolean     |
| Loop <code>d.items()</code>                      | <b>O(n)</b>         | Must visit every pair once               |

**Space:**  $O(n)$  — the dictionary holds one entry per key, plus some extra slots so hashing stays fast.

Two honest caveats. First, these are **average** times. In rare cases where many keys hash to the same slot (a *collision*), operations can degrade — but Python's implementation makes this extremely uncommon in practice. Second, the constant-time win isn't free: a dict uses a bit **more memory** than a list because it reserves spare room to keep collisions low. You trade memory for speed.

## 18. Alternative Solutions

**`collections.defaultdict`** — when you want missing keys to auto-create a default value (useful for counting or grouping):

```
from collections import defaultdict

counts = defaultdict(int)
for word in ["a", "b", "a"]:
    counts[word] += 1      # no KeyError on first sight of a word
# {'a': 2, 'b': 1}
```

**`collections.Counter`** — a specialized dict for tallying:

```
from collections import Counter
Counter(["a", "b", "a"])      # Counter({'a': 2, 'b': 1})
```

**Comparison:** A plain `dict` is the general tool. Use `defaultdict` to skip existence checks when building up values, and `Counter` when your whole job is counting. All three are dictionaries under the hood, so they share the same  $O(1)$  lookup.

## 19. Edge Cases

---

- **Empty dictionary:** `{}` is valid. `len({})` is `0`, and `"x" in {}` is `False`. Reading any key raises `KeyError` — use `get()`.
- **Missing key:** `d["nope"]` crashes; `d.get("nope")` returns `None`.
- **Duplicate keys at creation:** `{"a": 1, "a": 2}` silently keeps the **last** value → `{"a": 2}`.
- **Unhashable key:** using a list as a key raises `TypeError: unhashable type: 'list'`. Use a tuple instead.
- **Falsy values vs missing keys:** `d.get("k")` returns `None` for a missing key *and* for a key whose value is `None`. If you must distinguish them, use `"k" in d`.

## 20. Common Mistakes

---

1. **Indexing a key that might not exist.** `d[key]` crashes on absence. Reach for `d.get(key, default)` when the key is uncertain.
2. **Using a list as a key.** Keys must be immutable. `d[["a"]] = 1` raises `TypeError` — use a tuple: `d[("a",)] = 1`.
3. **Looping without `.items()`.** `for x in d:` gives only keys, not pairs. Use `for k, v in d.items():` when you need both.
4. **Expecting duplicate keys to coexist.** Assigning to an existing key overwrites it — the old value is gone, not stacked.
5. **Confusing "key missing" with "value is None."** `get()` can't tell them apart; use `in` when the difference matters.
6. **Assuming dicts sort themselves.** Modern Python preserves *insertion* order, not sorted order. Use `sorted()` if you need ordering.

## 21. Interview Questions

---

- Why is dictionary lookup  $O(1)$  while list search is  $O(n)$ ?
- What is a hash function, and why must dictionary keys be immutable?
- What happens on a hash collision, and how does Python handle it?
- When would you choose a list over a dictionary?
- What's the difference between `d[key]` and `d.get(key)` ?
- How would you count the frequency of items in a list using a dict?
- Is dictionary iteration order guaranteed in modern Python?

## 22. Similar Problems

---

- **Two Sum (LeetCode 1)** — the canonical dict-for-speed problem: store seen numbers as keys for  $O(1)$  complement lookup.
- **Group Anagrams (LeetCode 49)** — use a sorted-letter signature as a dict key to bucket words.
- **Valid Anagram (LeetCode 242)** — count characters with a dict/ `Counter` and compare.
- **Ransom Note (LeetCode 383)** — tally letter counts and check availability.
- **First Unique Character in a String (LeetCode 387)** — a frequency dict finds the first count of one.

Each one replaces a slow nested scan with a single-pass dictionary lookup — the exact pattern this lesson teaches.

## 23. Key Takeaways

---

- A dictionary stores values under **keys**, not positions, and finds them in roughly **one step** via hashing.
- Lookup, insert, and delete are  **$O(1)$**  on average, no matter the size.
- **Keys are unique and immutable**; assigning to an existing key overwrites it.
- Use `get()` to read safely, `.items()` to loop over pairs, and **nesting** to model real records.
- Reach for a **list** when order and position matter; reach for a **dict** when you look things up by name.

## 24. Watch the Video

---

Seeing the coat-check and locker analogies animated makes the "jump straight to the value" idea stick far better than reading alone. Watch the full walkthrough here: {LINK}



## 25. About the Series

---

This is part of the **Daily Python** series — short, focused lessons that turn core Python and **data structures** into intuition you can actually use in interviews and real projects. Each entry teaches one idea deeply: the *why* behind it, clean **Python** code, and the **coding interview** patterns that rely on it. Follow along daily and the fundamentals compound fast.

## 26. Call To Action

---

If this cleared up why dictionaries are so fast, **like the video on YouTube** and **subscribe** so you don't miss the next lesson. **Subscribe to the Substack** for the written deep-dives, **drop a comment** with the dict trick you use most, and **share** this with someone who's still scanning lists to find one record.

---

## Quick Review

---

### Trigger: need instant lookup by a unique label instead of scanning?

Pattern: dictionary (hash map). When you see 'look up value by key' or 'count/group by X', reach for a dict.

### Time complexity of dict get, set, and 'in' checks?

Average  $O(1)$  per operation. Worst case  $O(n)$  under heavy hash collisions, but effectively constant in practice.

### Space complexity of a dictionary with $n$ entries?

$O(n)$  — one slot per key-value pair, plus hash-table overhead for empty buckets.

### Trickiest bug: accessing a missing key?

`d[key]` raises `KeyError`. Use `d.get(key, default)` or ``if key in d`` to read safely without crashing.

### Dictionary vs list: when to pick which?

Dict for lookup by label/id ( $O(1)$ ); list for ordered items or index access. Membership: dict  $O(1)$ , list  $O(n)$ .

**Follow-up: why must dict keys be immutable?**

Keys must be hashable; lists/dicts can't be keys since mutation would change the hash. Use tuples, strings, or numbers.

---

Python Lesson 10: Dictionaries (Key-value storage, lookup, iteration, and nested data structures) — free Python cheat sheet